Fondements de l'Informatique
et
Structure des Ordinateurs

L'Algèbre de Boole, la Représentation de l'Information
et l'Architecture des Systèmes Numériques

Arthur LOUVET

Licence Informatique - Semestres 1 & 2 (2025-206)

Table des matières

I Algèbre de Boole et Fonctions Logiques 5

1 L'Algèbre de Boole 5
1.1 Le domaine de définition . . . 5
1.2 Les opérateurs de base . . . 5
1.2.1 Opérations unaires et binaires . . . 5
1.3 Propriétés fondamentales . . . 5
1.4 Règle de De Morgan . . . 6

2 Les autres opérateurs usuels 7
2.1 Le OU booléen (XOR) . . . 7
2.2 Le NON OU (NOR) . . . 7
2.3 Le NON ET (NAND) . . . 7

3 Fonctions logiques 9
3.1 Propriétés des opérateurs . . . 9
3.1.1 La commutativité . . . 9
3.1.2 L'associativité . . . 9
3.2 Groupes et opérateur complet . . . 9
3.2.1 L'implication . . . 9
3.2.2 Opérateur / Groupe et opérateur complet . . . 10

4 Les égalités de fonction 11
4.1 Vérification par table de vérité . . . 11
4.2 Normalisation de l'écriture d'une fonction . . . 11
4.3 La forme canonique : Passage entre la formule . . . 12

5 Fonctions incomplètes et états indéterminés 13
5.1 Principe . . . 13
5.2 Exemple de fonctions incomplètes . . . 13
5.3 Table de vérité avec états indéterminés . . . 13
5.4 Formes normales . . . 14

6 La simplification d'une fonction 14

II Représentation de l'Information 15

7 Les entiers naturels (taille infinie) $\mathbb{N}$ 15
7.1 Types de nombres . . . 15
7.2 Le codage . . . 15
7.3 Conversion : division euclidienne . . . 16
7.3.1 Principe général . . . 16

8 Systèmes de codage **17**
8.1 Rappel : système binaire . . . 17
8.2 Codes pondérés . . . 17
8.2.1 Code BCD (Binary Coded Decimal) . . . 17
8.2.2 Code de Aiken . . . 17
8.3 Codes non-pondérés . . . 17
8.3.1 Code excédant 3 (+3) . . . 17
8.3.2 Code de GRAY . . . 18

9 Codes redondants - Détection d'erreurs **20**
9.1 Bit de parité . . . 20
9.2 Code pondéré redondant . . . 20
9.3 Code ASCII . . . 20

III Circuits et Architecture des Ordinateurs **21**

10 Introduction à l'architecture des ordinateurs **21**
10.1 Composants de base . . . 21
10.2 Opérations binaires . . . 21

11 Les circuits : Transcodeurs **21**
11.1 Définition générale . . . 21
11.2 Le codeur . . . 22
11.3 Le décodeur . . . 22
11.4 Le transcodeur . . . 23

12 Circulation de l'information - Sur les routes de l'information **24**
12.1 Introduction . . . 24
12.2 Bus série et bus parallèle . . . 24
12.2.1 Bus série . . . 24
12.2.2 Bus parallèle . . . 24
12.2.3 Comparaison . . . 25
12.3 Aiguillage de l'information . . . 25
12.3.1 Autorisation de passage par porte ET . . . 25
12.3.2 Autorisation de passage par porte OU . . . 26

13 Le Multiplexeur **27**
13.1 Description du circuit . . . 27
13.1.1 Composants d'entrée . . . 27
13.1.2 Composants de sortie . . . 27
13.2 Fonctionnement . . . 27
13.3 Réalisation du circuit . . . 28
13.4 Applications des multiplexeurs . . . 28
13.4.1 Génération de fonction logique . . . 28
13.4.2 Conversion parallèle/série . . . 29

14 Le Démultiplexeur **30**
14.1 Description . . . 30
14.2 Composants . . . 30
14.3 Fonctionnement . . . 30
14.4 Réalisation . . . 30
14.5 Application : Conversion série/parallèle . . . 31

15 L'Unité Arithmétique et Logique (UAL/ALU) **32**
15.1 Présentation générale . . . 32
15.2 Architecture de l'ALU . . . 32
15.3 Les flags (drapeaux) . . . 32
15.3.1 Zero Flag (ZF) . . . 32
15.3.2 Sign Flag (SF) . . . 33
15.3.3 Carry Flag (CF) . . . 33
15.3.4 Overflow Flag (OF) . . . 34
15.3.5 Parity Flag (PF) . . . 34
15.4 Tableau récapitulatif des flags . . . 35
15.5 Utilisation des flags . . . 35

16 Conclusion et points clés **36**
16.1 Optimisation des circuits . . . 36
16.2 Choix du système de codage . . . 36
16.3 Architecture et circuits . . . 36
16.4 Circulation de l'information . . . 36
16.5 L'ALU et les flags . . . 36

Première partie
Algèbre de Boole et Fonctions Logiques

1 L'Algèbre de Boole

1.1 Le domaine de définition

Définition : Ensemble de définition des nombres booléens

L'ensemble de définition des nombres booléens est $\{0, 1\}$ mais ces nombres ne peuvent prendre que l'existence :

- $0 \rightarrow$ valeur fausse
- $1 \rightarrow$ valeur vraie

1.2 Les opérateurs de base

1.2.1 Opérations unaires et binaires

Définition : Opérations de base

Ces nombres utilisent 4 opérateurs :

- **NON** (non a, $-a$, $\overline{a}$) $\rightarrow$ opération unaire
  $a \neq b \Rightarrow 1 = 0$
- **OU** ($a \vee b$, $a + b$) $\rightarrow$ opération binaire
- **ET** ($a \wedge b$, $a \cdot b$) $\rightarrow$ opération binaire

1.3 Propriétés fondamentales

Propriétés fondamentales de l'algèbre de Boole

Ces nombres possèdent des propriétés fondamentales :

1. **Idempotence :** $a + a = a \quad / \quad a \cdot a = a$
2. **Involution :** $\overline{\overline{a}} = a$
3. **Commutativité :** $a + b = b + a \quad / \quad a \cdot b = b \cdot a$
4. **Élément neutre :** $a + 0 = a \quad / \quad a \cdot 1 = a$
5. **Élément absorbant :** $a + 1 = 1 \quad / \quad a \cdot 0 = 0$
6. **Complémentarité :** $a + \overline{a} = 1 \quad / \quad a \cdot \overline{a} = 0$
7. **Associativité :** $(a + b) + c = a + (b + c)$
   $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

Propriétés de distributivité

— **Distributivité de "$\vee$" sur "$\wedge$" :**

$$(a + b) \cdot c = (a \cdot c) + (b \cdot c)$$

— **Distributivité de "$\wedge$" sur "$\vee$" :**

$$(a \cdot b) + c = (a + c) \cdot (b + c)$$

1.4 Règle de De Morgan

Théorème : Lois de De Morgan

Pour tous $a, b \in \{0, 1\}$:

$$\overline{(a + b)} = \overline{a} \cdot \overline{b}$$

$$\overline{(a \cdot b)} = \overline{a} + \overline{b}$$

Ces lois permettent de transformer la négation d'une somme en produit de négations, et réciproquement.

2 Les autres opérateurs usuels

2.1 Le OU booléen (XOR)

Définition : OU exclusif (XOR)

Le OU booléen (XOR) noté $a \oplus b$ est différent du OU classique car il est seulement vrai lorsque les deux valeurs sont différentes. Pour que $a \oplus b$ soit vraie, il faut exactement l'une des deux valeurs soit vraie (mais pas les deux).

Table de vérité du OU exclusif

a	b	$a \vee b$	$a \oplus b$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

2.2 Le NON OU (NOR)

Définition : NOR

Le "non OU" (NOR) noté $\text{NOR}(a, b)$ est différent du OU classique car il est seulement vrai lorsque a et b sont faux (que celui-ci soit vrai).

Table de vérité du NOR

a	b	$a \vee b$	NOR
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

2.3 Le NON ET (NAND)

Définition : NAND

Le "non ET" (NAND) se note $\overline{a \cdot b}$ et est différent du ET car pour le ET, les valeurs doivent être vraies. Alors que pour le non ET, les valeurs ne doivent pas être vraies en même temps.

Table de vérité du NAND

a	b	$a \cdot b$	$\overline{a \cdot b}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

3 Fonctions logiques

3.1 Propriétés des opérateurs

3.1.1 La commutativité

Propriété : Rappel pour tous a

- $a + b = b + a$
- $a \cdot b = b \cdot a$

3.1.2 L'associativité

Propriété : Attention pour NOR et NAND

Attention : $\forall a, b, c \in \{0, 1\}$:

- $\overline{a + b + c} \neq (\overline{a + b}) + c$
- $\overline{a \cdot b \cdot c} \neq (\overline{a \cdot b}) \cdot c$

Ainsi, il n'y a aucune propriété / manière évidente en NOR et NAND (il n'y a pas de propriété d'associativité).

Remarque

On associe couramment le "NOR" à d'autres calculs mais le principe est le même !!!

3.2 Groupes et opérateur complet

3.2.1 L'implication

Définition : Implication

Voici la table de vérité de l'implication :

a	b	$a \Rightarrow b$	$b \Rightarrow a$
0	0	1	1
0	1	1	0
1	0	0	1
1	1	1	1

3.2.2 Opérateur / Groupe et opérateur complet

Définition : Groupe complet

L'ensemble $\{+, \cdot, -\}$ est un groupe complet.
L'ensemble $\{\oplus, -\}$ est-il complet ?

Pour tous $a, b \in \{0, 1\}$:

- $a \cdot b = \overline{a} \oplus b$ (pour le ET)
- $a + b = \overline{(\overline{a + b})} = \overline{(\overline{a} \cdot \overline{b})}$

Donc $\{\oplus, -\}$ est donc bel et bien un ensemble complet.

Propriété

Les fonctions logiques ou booléennes ($+$ pour canonique) :
Soit $f_1(a, b, c) = a + b$ et $f_2(a, b, c) = (a \cdot c) + (\overline{a} \cdot b) + a \cdot c$
$f_2(a, b, c) = \overline{a} \cdot b \cdot c + a \cdot \overline{b} + \overline{a} \cdot \overline{b} \cdot \overline{c}$

4 Les égalités de fonction

4.1 Vérification par table de vérité

Exemple avec table de vérité

Voici une table de vérité complète pour fonctions à 3 variables :

a	b	c	$\overline{a}$	$\overline{b}$	$a \cdot \overline{b}$	$\overline{a} \cdot \overline{b}$	$a \cdot b$	$c + a$	f_c
0	0	0	1	1	0	1	0	0	0
0	0	1	1	1	0	1	0	1	0
0	1	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	1	0
1	0	0	0	1	1	0	0	1	1
1	0	1	0	1	1	0	0	1	0
1	1	0	0	0	0	0	1	1	1
1	1	1	0	0	0	0	1	1	1

4.2 Normalisation de l'écriture d'une fonction

Définition : Formule normale

La formule normale se compose de deux formes :

Forme disjonctive :

- Les opérateurs conjonctifs ne portent que sur des variables
- Les variables sont regroupées par forme : conjonction
- Les termes sont regroupés : $\left[\frac{\text{OU}}{\text{ET}}\right]$ ou $\left[\frac{\text{OU}}{?}\right]$

Forme conjonctive :

- Les opérateurs disjonctifs ne portent que sur des variables
- Les variables sont regroupées par formes : disjonction
- Les termes sont regroupés : $\left[\frac{\text{ET}}{\text{OU}}\right]$ ou $\left[\frac{\text{ET}}{?}\right]$

Exemples :

- **FND :** $f(a, b, c) = (a \cdot \overline{b} \cdot c) + (\overline{a} \cdot b)$
- **FNC :** $f(a, b, c) = (a + b + \overline{c}) \cdot (b + c)$

4.3 La forme canonique : Passage entre la formule

Définition : Forme canonique

Forme canonique disjonctive :

$$f_1(a,b,c) = \overline{a}\overline{b}c + \overline{a}b\overline{c} + abc \quad \text{(disjonctive)}$$

Forme canonique conjonctive :

$$f_2(a,b,c) = (a+b+\overline{c}) \cdot (a+\overline{b}+c) \cdot (\overline{a}+\overline{b}+c) \quad \text{(conjonctive)}$$

Exemple : Table de vérité

a	b	c	f_1	f_2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Comment passer à une forme canonique ?

$$f_c = (a+b+c) \cdot (a+\overline{c}) \cdot (\overline{a}+\overline{b}+c)$$

5 Fonctions incomplètes et états indéterminés

5.1 Principe

Définition : États indéterminés

Pour la transformation d'états indéterminés, on s'en sert pour simplifier les fonctions en choisissant le 1 le plus souvent possible.
Lorsque l'on a plus de bits que nécessaires, ceux en trop sont dans un état indéterminé. C'est une **optimisation de bits et de place**.

Remarque importante

Voici pourquoi on simplifie les fonctions : pour les récupérer et les « économiser ».

5.2 Exemple de fonctions incomplètes

Soient les fonctions suivantes :

$$M_1 = V_1V_2 + V_3V_4 \tag{1}$$
$$M_2 = V_2V_4 + V_1V_3 \tag{2}$$
$$M_3 = V_3V_4 + V_2V_3 \tag{3}$$

5.3 Table de vérité avec états indéterminés

V_1	V_2	V_3	V_4	M_1	M_2	M_3
0	0	0	0	0	0	0
0	0	0	1	-	0	0
0	0	1	0	0	0	1
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	1	-	1	0
1	0	1	0	0	1	1
1	0	1	1	1	1	1
1	1	0	0	1	0	0
1	1	0	1	1	1	0
1	1	1	0	1	1	1
1	1	1	1	1	1	1

Table 1 – Table de vérité avec états indéterminés (notés « - »)

5.4 Formes normales

- **FND (Forme Normale Disjonctive)** : SOMME = 1
- **FNC (Forme Normale Conjonctive)** : SOMME = 0

6 La simplification d'une fonction

Soit :

$$f_1 = abc + b(a + \overline{c}) = abc + ab + b\overline{c}$$
$$f_1 = abc + abc + ab\overline{c} + ab\overline{c} + a\overline{b}\overline{c} + \overline{a}b\overline{c}$$
$$f_1 = abc + ab\overline{c} + abc + a\overline{b}\overline{c} = ab + b\overline{c}$$

Deuxième partie
Représentation de l'Information

7 Les entiers naturels (taille infinie) $\mathbb{N}$

7.1 Types de nombres

- Relatifs ∞ : $\mathbb{Z}$ (complément à la base)
- Décimaux ∞ : $\mathbb{D}$
- Rationnels : arithmétique infinie $\mathbb{Q}$
- Réels : $\mathbb{R}$
- Complexe : $\mathbb{C}$

7.2 Le codage

Définition : Codage N

Le codage $N = \sum_{i=0}^{k} n_i \cdot b^i$ où :

- k : nombre de poids de chiffres à gauche
- chiffres de gauche, rangés en groupe $k^{\text{ième}}$
- b : base de numération, ≥ 2 arbitraire (complément à la base)

Exemple : Langage européen (base 10)

$$566 = 5 \times 10^2 + 6 \times 10^1 + 6 \times 10^0$$

En base 9 :

$$(568)_9 = (5 \times 9^2 + 6 \times 9^1 + 8 \times 9^0) = 405 + 54 + 8 = (467)_{10}$$
$$(5685)_9 = (5 \times 9^3 + 6 \times 9^2 + 8 \times 9^1 + 5 \times 9^0) = (4172)_{10}$$

7.3 Conversion : division euclidienne

Exemple : Conversion de base

$$\text{Exemple : } (10111101)_2 = (189)_{10}$$

Si on veut convertir 585 en base 7, on utilise la division euclidienne :

$$\begin{array}{rcl} 585 & = & 7 \times 83 + 4 \\ 83 & = & 7 \times 11 + 6 \\ 11 & = & 7 \times 1 + 4 \\ 1 & = & 7 \times 0 + 1 \end{array}$$

Résultats : $585_{10} = (1464)_7$

7.3.1 Principe général

$$\begin{aligned} (N)_b &= \sum n_i \cdot b^i \\ N &= n_0 + b(n_1 + b(n_2 + b \cdot n_3 + \ldots)) \\ N &= b \times Q + R \end{aligned}$$

où R est le reste de la division (chiffre de poids faible) et Q est le quotient.

8 Systèmes de codage

8.1 Rappel : système binaire

Le binaire est un **code pondéré**. La formule binomiale permet de convertir un nombre binaire en décimal :

$$\text{Valeur décimale} = \sum_{i=0}^{n-1} B_i \times 2^i$$

où B_i est le bit de rang i.

8.2 Codes pondérés

8.2.1 Code BCD (Binary Coded Decimal)

Définition : Code BCD

- Code uniquement les 10 chiffres décimaux (0 à 9)
- Chaque chiffre décimal est représenté par 4 bits
- Code pondéré et réfléchi

8.2.2 Code de Aiken

Définition : Code de Aiken

Le code de Aiken possède les propriétés suivantes :

- Pratique
- Pondéré avec les poids : **2, 4, 2, 1**
- BCDique (code les 10 chiffres)
- Réfléchi
- Autocomplémentaire

8.3 Codes non-pondérés

8.3.1 Code excédant 3 (+3)

Définition : Code excédant 3

- Code BCDique
- On prend le code binaire naturel et on ajoute 3
- Exemple : $0 \to 0 + 3 = 3$ (0011), $1 \to 1 + 3 = 4$ (0100), etc.

Décimal	Poids				Code de Aiken
	2	4	2	1	
0	0	0	0	0	0000
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	0	1	1	0011
4	0	1	0	0	0100
5	1	0	1	1	1011
6	1	1	0	0	1100
7	1	1	0	1	1101
8	1	1	1	0	1110
9	1	1	1	1	1111

Table 2 – Table du code de Aiken

Décimal	Binaire naturel	Excédant 3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Table 3 – Table du code excédant 3

8.3.2 Code de GRAY

Définition : Code de GRAY

Le code de GRAY est un code cyclique très utilisé dans les encodeurs rotatifs et les dispositifs électromécaniques.

Principe de construction :

1. On commence à 0
2. Pour la valeur suivante, on inverse le bit le plus à droite
3. On ne retombe jamais sur une valeur déjà connue
4. Un seul bit change entre deux valeurs consécutives

Avantages :

— Évite les erreurs de lecture lors de transitions

— Particulièrement utile pour les roues/molettes électromécaniques

— Un seul bit change à la fois = moins d'erreurs de lecture

Décimal	Binaire naturel	Code de GRAY
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

TABLE 4 – Table du code de GRAY (4 bits)

Application pratique

Le code de GRAY est utilisé dans les encodeurs rotatifs (comme les molettes de souris, les potentiomètres numériques, etc.) car lors de la rotation, un seul segment change d'état à la fois, ce qui évite les erreurs de lecture dues aux désynchronisations mécaniques.

9 Codes redondants - Détection d'erreurs

9.1 Bit de parité

Définition : Bit de parité

Le bit de parité permet d'effectuer des tests de communication entre machines et de détecter des erreurs de transmission.
Pour déterminer la parité d'une valeur en binaire, on fait un **OU EXCLUSIF (XOR)** entre chaque valeur de manière cumulative (au fur et à mesure).

Exemple : Calcul de parité

Soit la valeur binaire : 1011

$$\begin{aligned}\text{Parité} &= 1 \oplus 0 \oplus 1 \oplus 1 \\ &= 1 \oplus 0 = 1 \\ &= 1 \oplus 1 = 0 \\ &= 0 \oplus 1 = 1\end{aligned}$$

Le bit de parité est donc 1 (parité impaire).

Application

Exemple : barrette mémoire avec détection d'erreurs utilisant ce fonctionnement.

9.2 Code pondéré redondant

Définition : Code redondant à 5 bits

Pour le code pondéré redondant, on utilise 5 bits ($2^5 = 32$ combinaisons possibles).
Propriété importante : chaque code de valeurs ne contient que des 1, donc s'il n'y a pas 2×1, on peut détecter une erreur.

9.3 Code ASCII

Définition : Code ASCII

Le code **ASCII** (American Standard Code for Information Interchange) est :

- Codé sur 2^7 bits (128 valeurs)
- $+$ 1 bit de parité (8 bits au total)
- ou ce bit utilisé pour un code ASCII étendu (variable selon les pays pour les caractères différents)

Troisième partie
Circuits et Architecture des Ordinateurs

10 Introduction à l'architecture des ordinateurs

10.1 Composants de base

Définition : Composants fondamentaux

Un ordinateur est composé des éléments fondamentaux suivants :

- **UAL (Unité Arithmétique et Logique)** : ALU en anglais, effectue les opérations arithmétiques et logiques
- **Unité de contrôle** : coordonne les opérations de l'ordinateur
- **Entrées / Sorties** : permettent la communication avec l'extérieur

10.2 Opérations binaires

Les opérations de base se font sur des opérandes binaires $\{0, 1\}$ avec les opérateurs : $+$, $-$, $\times$.

Ces opérations possèdent des propriétés et peuvent être représentées par des fonctions et des tables de vérité, qui permettent ensuite la simplification des circuits logiques.

11 Les circuits : Transcodeurs

11.1 Définition générale

Définition : Transcodeur

Dans un circuit, on a le **transcodeur** : circuit dans lequel on a :

- le **codeur** (pas de code $\rightarrow$ code)
- le **décodeur** (code $\rightarrow$ pas de code)

Le transcodeur fait donc : code $\rightarrow$ code

11.2 Le codeur

Définition : Codeur

Côté CODEUR : 1 et 1 seule entrée à 1

- "pas de code" : 4 valeurs, on a 4 touches : 0, 1, 2, 3
- Si on appuie sur la première touche, le circuit de codage doit générer 0, etc.
- On fait un circuit avec 4 entrées, 2 sorties
- Tableau avec $2^4 = 16$ lignes, 2 fonctions
- Toutes les autres valeurs que 0, 1, 2 et 3 ne nous intéressent pas (états indéterminés)

En sortie : le codeur sort la valeur qui nous intéresse, celle que l'on a mis à 1 (pression).

Exemple : Codeur 4 vers 2

Valeur	E_3	E_2	E_1	E_0	S_1	S_0
0	0	0	0	1	0	0
1	0	0	1	0	0	1
2	0	1	0	0	1	0
3	1	0	0	0	1	1

Les équations logiques sont :

$$S_1 = E_2 + E_3$$
$$S_0 = E_1 + E_3$$

11.3 Le décodeur

Définition : Décodeur

Côté DÉCODEUR :

- Prend en entrée 1 valeur codée dans un code
- Prend en sortie, la sortie activée dont le numéro correspond à la valeur d'entrée

Exemple de décodeur binaire sur 3 bits, 8 sorties.

Exemple : Décodeur 3 vers 8

E_2	E_1	E_0	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Les équations logiques sont :

$$S_0 = \overline{E_2} \cdot \overline{E_1} \cdot \overline{E_0}$$
$$S_1 = \overline{E_2} \cdot \overline{E_1} \cdot E_0$$
$$S_2 = \overline{E_2} \cdot E_1 \cdot \overline{E_0}$$
$$\text{etc...}$$

11.4 Le transcodeur

Définition : Transcodeur

Côté TRANSCODEUR :

- Prend en entrée 1 valeur dans 1 code
- Prend en sortie la même valeur dans un autre code

Exemple : code GRAY sur 3 bits → binaire

Exemple : Transcodeur GRAY vers Binaire

Val	G_2	G_1	G_0	B_2	B_1	B_0
0	0	0	0	0	0	0
1	0	0	1	0	0	1
2	0	1	1	0	1	0
3	0	1	0	0	1	1
4	1	1	0	1	0	0
5	1	1	1	1	0	1
6	1	0	1	1	1	0
7	1	0	0	1	1	1

Les équations de conversion sont :

$$B_2 = G_2$$
$$B_1 = G_2 \oplus G_1$$
$$B_0 = G_2 \oplus G_1 \oplus G_0$$

12 Circulation de l'information - Sur les routes de l'information

12.1 Introduction

Définition : Communication dans un processeur

Un processeur est composé de plusieurs circuits remplissant chacun leurs fonctions. Entre ces circuits, les informations **circulent** sur des **bus**.
Le terme **bus** désigne l'ensemble du moyen de transport, matériel et logiciel (protocole).

12.2 Bus série et bus parallèle

Le terme information désigne ici une valeur quelconque, codée sur un certain nombre de bits. La question de la simultanéité de la circulation des bits se pose.

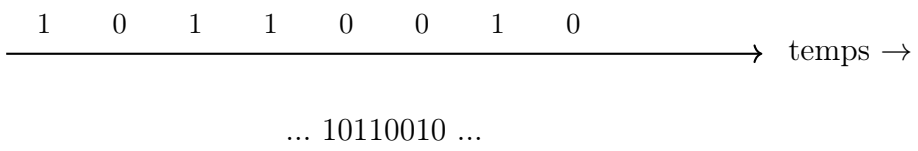
12.2.1 Bus série

Définition : Bus série

On parle de communication et de **bus série** quand les bits circulent **les uns après les autres** sur un même support.

Exemples : RS232, USB, SATA, Ethernet

Représentation d'un bus série



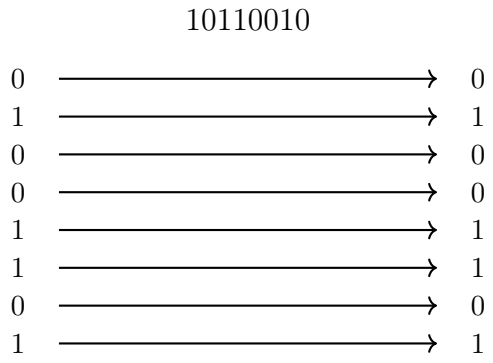
12.2.2 Bus parallèle

Définition : Bus parallèle

On parle de communication et de **bus parallèle** quand les bits circulent **ensemble** sur des supports séparés.

Exemples : bus mémoire, Centronic, PATA, SCSI, PCI

Représentation d'un bus parallèle



12.2.3 Comparaison

Critère	Bus Série	Bus Parallèle
Nombre de fils	Peu	Nombreux
Coût	Moins cher	Plus cher
Vitesse (théorique)	Plus lent	Plus rapide
Exemples	USB, SATA	DATA, PCI, IDE

TABLE 5 – Comparaison bus série vs bus parallèle

Remarque importante

Paradoxalement, avec les technologies modernes, les bus série sont devenus très rapides (USB 3.0, SATA III) et ont remplacé progressivement les bus parallèles qui souffraient de problèmes de synchronisation à haute fréquence.

12.3 Aiguillage de l'information

Principe de l'aiguillage

Il n'y a pas un bus par couple de circuits à connecter, mais souvent plusieurs circuits se partagent un bus pour échanger entre eux.
L'information qui circule sur un bus doit donc être :

- **Sélectionnée** (source)
- **Dirigée** (destination)

Il faut donc un moyen de faire passer ou d'empêcher le passage d'une information sur un fil.

12.3.1 Autorisation de passage par porte ET

On peut empêcher le passage d'une information sur un fil en utilisant une porte ET, en disant que l'absence d'information correspond à la valeur 0.

Notations :

- E : information en entrée
- S : information en sortie

— C : commande de passage

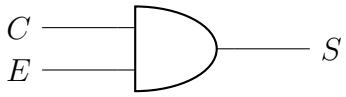
C	E	$S = C \cdot E$
0	0	0
0	1	0
1	0	0
1	1	1

Interprétation

C	S
0	0
1	E

— Si $C = 0$ alors la sortie S vaut 0 quelle que soit la valeur de E

— Si $C = 1$ alors la sortie S vaut E



12.3.2 Autorisation de passage par porte OU

On peut empêcher le passage d'une information sur un fil en utilisant une porte OU, en disant que l'absence d'information correspond à la valeur 1.

C	E	$S = C + E$
0	0	0
0	1	1
1	0	1
1	1	1

Interprétation

C	S
0	E
1	1

— Si $C = 1$ alors la sortie S vaut 1 quelle que soit la valeur de E

— Si $C = 0$ alors la sortie S vaut E

13 Le Multiplexeur

13.1 Description du circuit

Définition : Multiplexeur (MUX)

Un **multiplexeur** est un circuit permettant de faire passer sur un bus l'information en provenance d'un autre bus choisi parmi plusieurs.
Pour sélectionner le bus source, il faut lui fournir son numéro ou, en terme plus approprié, son **adresse**.

13.1.1 Composants d'entrée

Le multiplexeur comprend en entrée :

— N **bus de données** identiques, comportant le même nombre de "fils", nommés $B_0, B_1, \ldots, B_{N-1}$
Les fils du bus B_i seront eux aussi numérotés : $B_{i,0}, B_{i,1}, \ldots, B_{i,M-1}$, où M est le nombre de fils de chacun des bus.

— Un **bus de sélection** ou **bus d'adresse** de n fils tel que $N = 2^n$, nommé A

— Une **commande d'activation** CS (Chip Select)

13.1.2 Composants de sortie

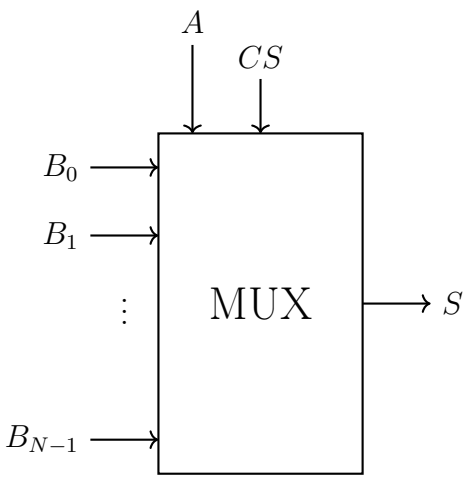
En sortie, un bus de données identique aux bus de données d'entrée, nommé S, comprenant M fils eux-mêmes nommés $S_0, S_1, \ldots, S_{M-1}$.

13.2 Fonctionnement

Règle de fonctionnement

— Si $CS = 0$ alors $S = 0$

— Si $CS = 1$ et $A = (i)_2$ alors $S = B_i$



13.3 Réalisation du circuit

Il faut envoyer vers la sortie S l'information portée par le bus dont le numéro i est mis sur le bus d'adresse. Cette adresse va être décodée par un décodeur.

Exemple : Multiplexeur 4 vers 1 (8 fils)

Pour un multiplexeur de 4 bus de 8 fils de large :

- Pour choisir parmi 4 bus il faut 4 valeurs
- Donc 2 bits d'adresse
- Donc un décodeur 2×4
- Les 4 bus d'entrée et le bus de sortie ont 8 fils chacun

Le schéma utilise des portes ET pour chaque fil, activées par les sorties du décodeur qui analyse l'adresse, puis une porte OU pour combiner les résultats.

13.4 Applications des multiplexeurs

13.4.1 Génération de fonction logique

Principe

Une fonction logique définit une valeur binaire pour toutes les combinaisons d'un ensemble de variables binaires.
Si on présente ces valeurs en entrée d'un multiplexeur à un bit de donnée, et qu'on présente les variables sur le bus d'adresse, on retrouve en sortie la valeur de la fonction.

Exemple : Fonction à 3 variables

Soit la fonction :

$$f_2(a,b,c) = \overline{a} \cdot \overline{b} \cdot \overline{c} + \overline{a} \cdot b \cdot \overline{c} + a \cdot \overline{b} \cdot \overline{c} + a \cdot \overline{b} \cdot c + a \cdot b \cdot c$$

a	b	c	f_2
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

En utilisant un multiplexeur à 3 bits d'adresse avec les constantes 1,0,1,0,1,1,0,1 en entrée, on obtient un circuit équivalent à la fonction.
Intérêt : La fonction est paramétrable par les constantes présentées en entrée du multiplexeur.

13.4.2 Conversion parallèle/série

Principe

La coexistence dans un ordinateur de plusieurs types de bus rend nécessaire diverses conversions.
Pour sérialiser une donnée fournie par un bus parallèle de N bits, il faut :

- Un multiplexeur à N bus d'entrée de 1 bit de donnée
- Envoyer la donnée sur les bus d'entrée, 1 bit par bus
- Envoyer successivement sur le bus d'adresse les valeurs croissantes à partir de 0

Les bits de la donnée se trouveront successivement sur la sortie.

14 Le Démultiplexeur

14.1 Description

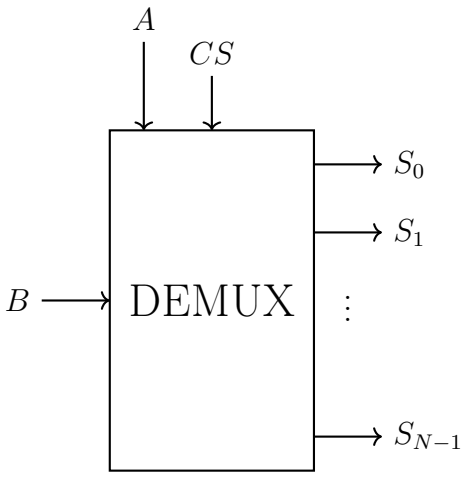
Définition : Démultiplexeur (DEMUX)

Un **démultiplexeur** est le circuit réalisant l'opération inverse du multiplexeur. Il prend une information sur un bus d'entrée et la dirige vers l'un des N bus de sortie selon l'adresse fournie.

14.2 Composants

Ce circuit comporte :

- Un **bus en entrée** de M fils parallèles, nommé B
- N **bus de sortie** identiques au bus d'entrée, nommés $S_0, S_1, \ldots, S_{N-1}$
- Un **bus d'adresse** de n bits tel que $N = 2^n$, nommé A
- Une **commande d'activation** CS



14.3 Fonctionnement

Règle de fonctionnement

- Si $CS = 0$ alors toutes les sorties valent 0 : $S_i = 0, \forall i$
- Si $CS = 1$ et $A = (i)_2$ alors $S_i = B$ et $S_j = 0, \forall j \neq i$

14.4 Réalisation

Le principe est similaire au multiplexeur mais inversé :

- Un décodeur analyse l'adresse A
- Chaque sortie du décodeur active un ensemble de portes ET
- Seul le bus de sortie correspondant à l'adresse reçoit l'information

14.5 Application : Conversion série/parallèle

Le démultiplexeur permet de convertir une information série en parallèle :

— L'information série arrive bit par bit sur l'entrée B

— L'adresse change séquentiellement

— Chaque bit est dirigé vers sa position dans le bus parallèle de sortie

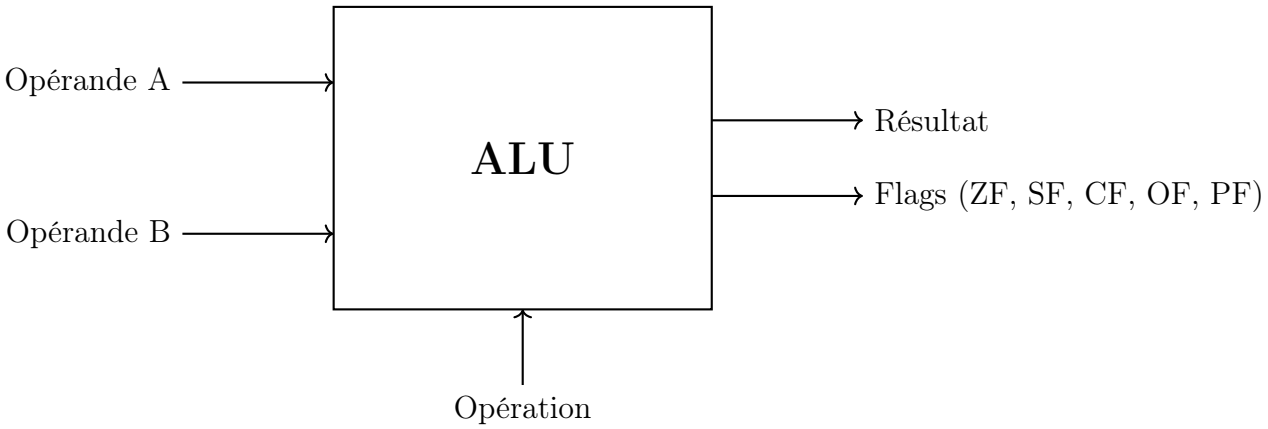
15 L'Unité Arithmétique et Logique (UAL/ALU)

15.1 Présentation générale

Définition : ALU

L'**Unité Arithmétique et Logique** (UAL en français, ALU en anglais pour *Arithmetic Logic Unit*) est le composant du processeur qui effectue les opérations arithmétiques et logiques.
Elle prend en entrée des opérandes et produit un résultat ainsi que des **flags** (drapeaux) indiquant des informations sur le résultat de l'opération.

15.2 Architecture de l'ALU



15.3 Les flags (drapeaux)

Les flags sont des bits qui donnent des informations sur le résultat de l'opération effectuée par l'ALU. Ils sont essentiels pour les instructions conditionnelles et la gestion des erreurs.

15.3.1 Zero Flag (ZF)

Définition : Zero Flag (ZF)

Le **Zero Flag** indique si le résultat de l'opération est nul.

$$ZF = \begin{cases} 1 & \text{si le résultat} = 0 \\ 0 & \text{sinon} \end{cases}$$

Formule logique : $ZF = \overline{a_0 + a_1 + a_2 + \ldots + a_n}$
où a_i sont les bits du résultat (on fait un OU de tous les bits, puis on nie).

Exemple

- $5 - 5 = 0 \Rightarrow \text{ZF} = 1$
- $7 - 3 = 4 = 0100_2 \Rightarrow \text{ZF} = 0$

15.3.2 Sign Flag (SF)

Définition : Sign Flag (SF)

Le **Sign Flag** indique le signe du résultat en représentation complément à deux.

$$SF = \text{bit de poids fort du résultat}$$

- $\text{SF} = 0 \Rightarrow$ résultat positif
- $\text{SF} = 1 \Rightarrow$ résultat négatif

Exemple (sur 8 bits)

- $00101100_2 = 44_{10} \Rightarrow \text{SF} = 0$ (positif)
- $11010011_2 = -45_{10}$ (C2) $\Rightarrow \text{SF} = 1$ (négatif)

15.3.3 Carry Flag (CF)

Définition : Carry Flag (CF)

Le **Carry Flag** indique s'il y a eu une **retenue** (carry) ou un **emprunt** (borrow) lors de l'opération sur le bit de poids fort.

- Pour une addition : CF = 1 s'il y a retenue sortante
- Pour une soustraction : CF = 1 s'il y a emprunt

Le CF est utilisé pour l'arithmétique **non signée**.

Exemple : Addition (sur 4 bits)

$$\begin{array}{rrl} & 1011_2 & (11_{10}) \\ + & 0111_2 & (7_{10}) \\ \hline & 1\,0010_2 & (2_{10} \text{ sur 4 bits}) \end{array}$$

Retenue sortante $\Rightarrow$ CF = 1
Le résultat devrait être 18 mais sur 4 bits on obtient 2 (débordement non signé).

15.3.4 Overflow Flag (OF)

Définition : Overflow Flag (OF)

L'**Overflow Flag** indique un **débordement en arithmétique signée** (complément à deux).
Il signale que le résultat est trop grand (ou trop petit) pour être représenté correctement dans le nombre de bits disponible.
Règle de détection :

$$OF = C_{in}(\text{MSB}) \oplus C_{out}(\text{MSB})$$

où C_{in} est la retenue entrante sur le bit de poids fort et C_{out} la retenue sortante.

Exemple : Addition signée (sur 4 bits)

$$\begin{array}{rll} & 0110_2 & (+6_{10}) \\ + & 0101_2 & (+5_{10}) \\ \hline & 1011_2 & (-5_{10} \text{ en C2, mais devrait être } +11) \end{array}$$

Deux nombres positifs donnent un résultat négatif $\Rightarrow$ OF $= 1$
Le résultat est incorrect en arithmétique signée.

Différence CF vs OF

- **CF** : pour l'arithmétique **non signée** (entiers naturels)
- **OF** : pour l'arithmétique **signée** (entiers relatifs en C2)

15.3.5 Parity Flag (PF)

Définition : Parity Flag (PF)

Le **Parity Flag** indique la **parité** du nombre de bits à 1 dans le résultat.

$$PF = \bigoplus_{i=0}^{n-1} a_i$$

(XOR de tous les bits du résultat)

- PF $= 0 \Rightarrow$ nombre pair de bits à 1 (parité paire)
- PF $= 1 \Rightarrow$ nombre impair de bits à 1 (parité impaire)

Exemple

- 10110010_2 : 4 bits à 1 $\Rightarrow$ PF = 0 (pair)
- 10110110_2 : 5 bits à 1 $\Rightarrow$ PF = 1 (impair)

Calcul pour 1011_2 :

$$PF = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

15.4 Tableau récapitulatif des flags

Flag	Nom	Signification
ZF	Zero Flag	Résultat = 0
SF	Sign Flag	Signe du résultat (bit de poids fort)
CF	Carry Flag	Retenue/emprunt (arithmétique non signée)
OF	Overflow Flag	Débordement (arithmétique signée)
PF	Parity Flag	Parité du résultat (XOR des bits)

Table 6 – Les principaux flags de l'ALU

15.5 Utilisation des flags

Les flags sont utilisés par les instructions conditionnelles pour prendre des décisions :

Exemples d'utilisation

- **JZ** (Jump if Zero) : sauter si ZF = 1
- **JS** (Jump if Sign) : sauter si SF = 1 (négatif)
- **JC** (Jump if Carry) : sauter si CF = 1
- **JO** (Jump if Overflow) : sauter si OF = 1
- **JP** (Jump if Parity) : sauter si PF = 1 (parité impaire)
- **JNZ** (Jump if Not Zero) : sauter si ZF = 0

Application pratique

Les flags permettent au processeur de :

- Détecter les débordements arithmétiques
- Implémenter des boucles (comparaison avec ZF)
- Gérer les nombres signés et non signés
- Détecter des erreurs de transmission (PF)
- Prendre des décisions conditionnelles dans le code machine

16 Conclusion et points clés

16.1 Optimisation des circuits

L'utilisation des états indéterminés permet de :

— Simplifier les fonctions logiques
— Économiser des bits
— Réduire la complexité des circuits
— Optimiser l'espace physique sur les circuits intégrés

16.2 Choix du système de codage

Le choix du système de codage dépend de l'application :

— **BCD/Aiken** : interfaçage avec des afficheurs, calculs décimaux
— **Code de GRAY** : encodeurs rotatifs, compteurs fiables
— **Excédant 3** : détection d'erreurs, arithmétique
— **ASCII** : représentation de caractères et texte

16.3 Architecture et circuits

Les circuits de transcodage (codeur, décodeur, transcodeur) sont essentiels pour :

— Interfacer différents systèmes
— Optimiser la représentation de l'information
— Faciliter les calculs et opérations
— Permettre la détection d'erreurs

16.4 Circulation de l'information

Les multiplexeurs et démultiplexeurs permettent :

— De partager efficacement les bus entre plusieurs circuits
— De convertir entre formats série et parallèle
— De générer des fonctions logiques configurables
— D'optimiser le routage des données dans le processeur

16.5 L'ALU et les flags

L'ALU est le cœur du processeur pour les opérations arithmétiques et logiques :

— Les flags fournissent des informations cruciales sur les résultats
— ZF, SF, CF, OF et PF permettent la gestion des conditions
— Distinction importante entre arithmétique signée (OF) et non signée (CF)
— Les flags sont la base des instructions conditionnelles

Ce document présente les fondements de l'algèbre de Boole, des fonctions logiques, de la représentation de l'information, de la circulation de l'information dans les circuits et de l'architecture des systèmes numériques.

Arthur LOUVET - Licence Informatique
Semestres 1 & 2 (2024-2025)